

End-to-End Data Platform Design

B2B Ticketing Platform Architecture & Implementation

Stack: Neon (Postgres), GitHub, Snowflake, dbt, Apache Airflow, Power BI

1. Executive Architecture Overview

This document details an enterprise-grade Modern Data Stack (MDS) implementation for a B2B ticketing platform. The design guarantees data integrity, scalability, and restartability using ELT principles.

Layer	Technology	Purpose
Source	Neon & GitHub	Neon (Serverless Postgres) holds transactional DB state; GitHub acts as the flat-file repository for reseller CSVs.
Orchestration (EL)	Apache Airflow	Extracts data from sources, schedules jobs, manages retries, and coordinates the load into Snowflake.
Data Warehouse	Snowflake	Highly scalable cloud data platform serving as the centralized repository (Raw, Staging, Analytics layers).
Transformation (T)	dbt (Data Build Tool)	Executes SQL-first transformations, dimensional modeling (Star Schema), and automated data quality tests.
Visualization	Power BI	Connects to Snowflake's Analytics layer to deliver business reporting and semantic modeling.

2. Apache Airflow Deep Dive

Apache Airflow is the "conductor of the orchestra." It does not process the data itself (it is not a compute engine like Spark); rather, it instructs other systems (Neon, Snowflake, dbt) on what to do and when to do it.

2.1 Where exactly do we use Airflow?

- **Data Ingestion (EL):** Triggering the extraction of data from Neon (via Postgres hooks) and GitHub (via HTTP requests), and executing the load commands into Snowflake's `RAW` schema.
- **Transformation Orchestration:** Triggering `dbt run` commands via the `BashOperator` or `DbtCloudOperator` immediately after ingestion completes.
- **Data Quality Checks:** Running `dbt test` to ensure data integrity before making it available to Power BI.

- **Alerting & Monitoring:** Sending notifications (e.g., to Slack or Email) if an ingestion fails, a CSV is missing on GitHub, or data quality tests fail.

2.2 Step-by-Step Airflow Implementation

1. **Environment Setup:** Deploy Airflow using Managed Workflows for Apache Airflow (MWAA), Astronomer, or a Docker Compose stack.
2. **Connection Configuration:** In the Airflow UI, configure connections:
 - `neon_postgres` : Connection to your Neon serverless DB.
 - `snowflake_default` : Connection to Snowflake with a dedicated service account.
 - `github_http` : HTTP connection for fetching raw CSVs.
3. **DAG Development:** Create Directed Acyclic Graphs (DAGs) using Python. Use standard Operators (`PythonOperator`, `SnowflakeOperator`) to define tasks.
4. **Idempotency & Restartability:** Design tasks so they can be re-run safely. For example, use Snowflake's `COPY INTO` or `MERGE` commands which handle duplicates automatically if a DAG fails mid-way and is restarted.

Airflow DAG Code (b2b_ticketing_dag.py)

This robust DAG includes error handling, dummy operators for flow control, and task dependencies.

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator
from datetime import datetime, timedelta
import pandas as pd
import io
import requests

default_args = {
    'owner': 'data_engineering',
    'depends_on_past': False,
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

def fetch_github_csv(**context):
    # Fetches CSV from Github and stages it via Pandas/SQLAlchemy
    url = "https://raw.githubusercontent.com/org/repo/data/DailySales_02152020_101.CSV"
    response = requests.get(url)
    response.raise_for_status() # Fail task if HTTP error

    df = pd.read_csv(io.StringIO(response.text))
    df['source_filename'] = "DailySales_02152020_101.CSV"
    df['ingestion_timestamp'] = pd.Timestamp.now()

    # Normally, you would use snowflake.connector.pandas_tools
    # to efficiently write df to Snowflake RAW schema here.
    print(f"Loaded {len(df)} rows to Snowflake RAW.")

with DAG('b2b_sales_elt_pipeline',
        default_args=default_args,
        schedule_interval='@daily',
        start_date=datetime(2023, 1, 1),
        catchup=False) as dag:

    start = EmptyOperator(task_id='start')

    # Task 1: Ingest Neon Data
    # In practice, use PostgresToSnowflakeOperator or a custom extract script
    ingest_neon_data = EmptyOperator(task_id='ingest_neon_postgres_to_snowflake')

    # Task 2: Ingest Github CSV
    ingest_github_csv = PythonOperator(
        task_id='ingest_github_csv_to_snowflake',
        python_callable=fetch_github_csv
    )

    wait_for_ingestion = EmptyOperator(task_id='wait_for_ingestion')

```

```

# Task 3: dbt Staging (Standardize Data)
dbt_run_staging = BashOperator(
    task_id='dbt_run_staging',
    bash_command='dbt run --select tag:staging --profiles-dir /path/to/profiles'
)

# Task 4: dbt Analytics (Build Star Schema)
dbt_run_analytics = BashOperator(
    task_id='dbt_run_analytics',
    bash_command='dbt run --select tag:analytics --profiles-dir /path/to/
profiles'
)

# Task 5: dbt Tests (Data Quality)
dbt_test = BashOperator(
    task_id='dbt_test',
    bash_command='dbt test --profiles-dir /path/to/profiles'
)

end = EmptyOperator(task_id='end')

# Define DAG Dependencies
start >> [ingest_neon_data, ingest_github_csv] >> wait_for_ingestion
wait_for_ingestion >> dbt_run_staging >> dbt_run_analytics >> dbt_test >> end

```

3. Data Transformation (dbt on Snowflake)

dbt applies software engineering practices to SQL. It handles the "T" in ELT by creating layered data models.

3.1 dbt Project Structure

- `models/staging/` : Cleans raw data, renames columns, casts types.
- `models/analytics/` : Contains Fact and Dimension tables (business logic).
- `models/schema.yml` : Defines tests (`not_null`, `unique`) and documentation.

3.2 Staging Model (Extracting Metadata from CSV filename)

```
-- models/staging/stg_github_sales.sql
{{ config(materialized='view', tags=['staging']) }}

WITH raw_data AS (
    SELECT * FROM {{ source('raw_db', 'raw_github_sales') }}
)
SELECT
    -- Cast and clean core fields
    transaction_id::VARCHAR AS source_transaction_id,
    event_name::VARCHAR AS event_name,
    event_type::VARCHAR AS event_type,
    ticket_qty::NUMBER(38,0) AS quantity,
    total_amount::NUMBER(10,2) AS total_amount,
    LOWER(sales_channel::VARCHAR) AS sales_channel,
    region::VARCHAR AS region,
    'RESELLER' AS sold_by_type,

    -- Regex/Split logic to parse 'DailySales_02152020_101.CSV'
    SPLIT_PART(REPLACE(source_filename, '.CSV', ''), '_', 3)::NUMBER AS seller_id,
    TRY_TO_DATE(SPLIT_PART(source_filename, '_', 2), 'MMDDYYYY') AS sale_date,

    ingestion_timestamp

FROM raw_data
WHERE transaction_id IS NOT NULL
```

3.3 Analytics Model (Fact Table Construction)


```

-- models/analytics/fct_sales.sql
{{ config(
    materialized='incremental',
    unique_key='sale_sk',
    tags=['analytics']
) }}

WITH neon_sales AS (
    SELECT * FROM {{ ref('stg_neon_sales') }}
),
github_sales AS (
    -- Join github data with neon events to get standard event_id
    SELECT
        g.*, e.event_id
    FROM {{ ref('stg_github_sales') }} g
    LEFT JOIN {{ ref('stg_neon_events') }} e ON g.event_name = e.event_name
),
combined_sales AS (
    SELECT source_transaction_id, sale_date, seller_id, sold_by_type, event_id,
    sales_channel, quantity, total_amount FROM neon_sales
    UNION ALL
    SELECT source_transaction_id, sale_date, seller_id, sold_by_type, event_id,
    sales_channel, quantity, total_amount FROM github_sales
)

SELECT
    {{ dbt_utils.generate_surrogate_key(['c.source_transaction_id', 'c.sold_by_type']) }}
AS sale_sk,
    c.source_transaction_id,
    c.sale_date,
    c.sales_channel,
    c.quantity,
    c.total_amount,

    -- Business Logic: Calculate Commission
    COALESCE(c.total_amount * s.commission_rate, 0) AS commission_amount,

    -- Surrogate Keys for Dimensions
    s.seller_sk,
    e.event_sk,
    TO_CHAR(c.sale_date, 'YYYYMMDD')::NUMBER AS date_id

FROM combined_sales c
LEFT JOIN {{ ref('dim_seller') }} s
    ON c.seller_id = s.seller_id AND c.sold_by_type = s.seller_type
LEFT JOIN {{ ref('dim_event') }} e
    ON c.event_id = e.event_id

{% if is_incremental() %}
    -- This ensures idempotency: only process new records on subsequent runs
    WHERE c.sale_date >= (SELECT MAX(sale_date) FROM {{ this }})
{% endif %}

```

3.4 dbt Testing (schema.yml)

```
version: 2
models:
  - name: fct_sales
    description: "Fact table combining all sales from Neon and Reseller CSVs"
    columns:
      - name: sale_sk
        tests:
          - unique
          - not_null
      - name: total_amount
        tests:
          - not_null
      - name: sales_channel
        tests:
          - accepted_values:
              values: ['web', 'mobile app', 'on-site']
```

4. Power BI Reporting & Semantic Layer

Power BI should connect to Snowflake via DirectQuery (if real-time is strictly needed) or Import Mode (recommended for performance, scheduled via Power BI Service after the Airflow DAG completes).

4.1 Advanced DAX Measures

To fulfill the specific business requirements (YoY comparisons dynamically filtered by Resellers/Event Types), use these DAX formulas in the Power BI Semantic Model:

```
// 1. Total Revenue
Total_Revenue = SUM('fct_sales'[total_amount])

// 2. Revenue for Last Year (Context Aware)
Revenue_LY = CALCULATE(
    [Total_Revenue],
    SAMEPERIODLASTYEAR('dim_date'[full_date])
)

// 3. Year over Year Growth %
YoY_Growth_% =
VAR variance = [Total_Revenue] - [Revenue_LY]
RETURN DIVIDE(variance, [Revenue_LY], 0)

// 4. Total Commission Paid
Total_Commission = SUM('fct_sales'[commission_amount])
```

4.2 Recommended Dashboard Layout (Theme: Corporate Indigo/Slate)

Go to Power BI > View > Customize Current Theme. Use a palette of Indigo (#4f46e5), Teal (#0d9488), and Slate Gray (#64748b). Apply a light gray background (#f8f9fc) to the page canvas, and use white (ffffff) backgrounds with 8px border radii for the visual containers.

Page 1: Sales Performance & YoY Comparison

- **Filter Pane (Left):** Slicers for `dim_date[Year/Month]`, `dim_seller[Region]`, `dim_event[Event Type]`.
- **KPI Cards (Top):** *Total Revenue, Tickets Sold, YoY Growth %.*
- **Line & Clustered Column Chart (Middle Left):**
 - X-Axis: `dim_date[week_of_year]`
 - Column Y-Axis: `Total_Revenue` (Current Year)
 - Line Y-Axis: `Revenue_LY` (Previous Year)
- **Matrix Visual (Bottom - The specific Feb '19 vs '20 req):**
 - Rows: `dim_seller[seller_name]`, `dim_event[event_type]`
 - Values: `Revenue_LY` (when filtered to Feb 2020, this shows Feb 2019), `Total_Revenue`, `YoY_Growth_%`

Page 2: Reseller Commissions & Event Popularity

- **Scatter Chart (Commission vs Sales):**

- X-Axis: `dim_seller[commission_rate]` (Do not summarize)
- Y-Axis: `Total_Revenue`
- Details: `dim_seller[seller_name]`

- **Top N Bar Chart (Most Popular Tickets per Region):**

- Y-Axis: `dim_event[event_name]`
- X-Axis: `Sum of Tickets Sold`
- Small Multiples (or Legend): `dim_seller[Region]`
- Filter: Apply a Top N filter (Top 3) on `event_name` based on Tickets Sold.